

Week 9: Production Operations — Monitoring, Security, and Backup

Phase 3: Internals & Operations | Week 9 of 12

Week Overview

This week covers running PostgreSQL in production: monitoring health metrics, implementing security hardening, executing backup and recovery procedures, and tuning configuration for your workload. These are the skills that separate "knows SQL" from "can operate a database in production."

Focus: A production database that you cannot monitor, secure, and recover is a liability. These skills protect your company.

Learning Objectives

By the end of this week, you will be able to:

- Monitor key PostgreSQL metrics using `pg_stat_*` views.
 - Identify bloat, slow queries, lock waits, and connection exhaustion.
 - Configure `pg_hba.conf` and `postgresql.conf` for security and performance.
 - Implement role-based access control and Row-Level Security.
 - Perform and verify `pg_dump`, `pg_restore`, and `pg_basebackup`.
 - Implement Point-in-Time Recovery (PITR) using WAL archiving.
 - Configure connection pooling with PgBouncer.
 - Use Prometheus + `postgres_exporter` for metric collection.
-

Required Reading

- `12_Production_PostgreSQL/` — All files
 - `14_Security/` — All files
 - `15_Backup_Recovery/` — All files
-

Daily Schedule

Monday — Monitoring and Health Metrics (60 min)

Topics:

- Essential `pg_stat_*` views every DBA uses daily
- Connection monitoring and `max_connections` tuning
- Lock monitoring: `pg_locks` and `pg_blocking_pids`
- Bloat detection: tables and indexes
- Long-running query detection and termination
- `pg_stat_activity` : what's happening right now

```

-- Connection overview
SELECT count(*) AS total, state, wait_event_type, wait_event
FROM pg_stat_activity
GROUP BY state, wait_event_type, wait_event
ORDER BY total DESC;

-- Long-running queries (> 5 minutes)
SELECT pid, now() - pg_stat_activity.query_start AS duration,
       query, state
FROM pg_stat_activity
WHERE (now() - pg_stat_activity.query_start) > INTERVAL '5 minutes'
      AND state != 'idle';

-- Block waiting queries
SELECT blocked_locks.pid AS blocked_pid,
       blocked_activity.username AS blocked_user,
       blocking_locks.pid AS blocking_pid,
       blocking_activity.username AS blocking_user,
       blocked_activity.query AS blocked_statement
FROM pg_catalog.pg_locks blocked_locks
JOIN pg_catalog.pg_stat_activity blocked_activity
  ON blocked_activity.pid = blocked_locks.pid
JOIN pg_catalog.pg_locks blocking_locks
  ON blocking_locks.locktype = blocked_locks.locktype
  AND blocking_locks.granted
  AND NOT blocked_locks.granted
JOIN pg_catalog.pg_stat_activity blocking_activity
  ON blocking_activity.pid = blocking_locks.pid;

-- Table bloat estimate
SELECT schemaname, tablename,
       pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS
total_size,
       n_dead_tup, n_live_tup,
       ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 1) AS dead_pct
FROM pg_stat_user_tables
WHERE n_dead_tup > 1000
ORDER BY n_dead_tup DESC;

-- Cache hit rate (should be > 99%)
SELECT sum(heap_blks_read) AS heap_read,
       sum(heap_blks_hit) AS heap_hit,
       ROUND(100.0 * sum(heap_blks_hit)
            / NULLIF(sum(heap_blks_hit) + sum(heap_blks_read), 0), 2) AS
cache_hit_pct
FROM pg_statio_user_tables;

-- Index usage ratio
SELECT relname, idx_scan, seq_scan,
       ROUND(100.0 * idx_scan / NULLIF(idx_scan + seq_scan, 0), 1) AS idx_use_pct

```

```
FROM pg_stat_user_tables
ORDER BY seq_scan DESC;
```

Tuesday — Security: Roles, Permissions, RLS (90 min)

Topics:

- CREATE ROLE, CREATE USER, GRANT, REVOKE
- Superuser, CREATEDB, CREATEROLE privileges
- Schema-level and table-level permissions
- Row-Level Security (RLS) policies
- `pg_hba.conf` : authentication methods (md5, scram, peer, cert)
- SSL/TLS configuration
- Least privilege principle in practice

```
-- Create application-specific roles
CREATE ROLE app_readonly;
CREATE ROLE app_readwrite;
CREATE ROLE app_admin;

-- Grant schema access
GRANT USAGE ON SCHEMA public TO app_readonly, app_readwrite, app_admin;

-- Table-level permissions
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO
app_readwrite;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO app_admin;

-- Future tables inherit permissions
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT ON TABLES TO app_readonly;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_readwrite;

-- Create application user
CREATE USER webapp_user WITH PASSWORD 'securepassword123!';
GRANT app_readwrite TO webapp_user;

-- Readonly reporting user
CREATE USER reporting_user WITH PASSWORD 'reportingpass456!';
GRANT app_readonly TO reporting_user;

-- Row-Level Security
CREATE TABLE tenant_data (
    id          SERIAL PRIMARY KEY,
    tenant_id   INTEGER NOT NULL,
    data        TEXT
);

ALTER TABLE tenant_data ENABLE ROW LEVEL SECURITY;
```

```
-- Policy: users can only see their own tenant's rows
CREATE POLICY tenant_isolation ON tenant_data
    USING (tenant_id = current_setting('app.tenant_id')::INTEGER);

-- Test RLS
SET app.tenant_id = '1';
SELECT * FROM tenant_data; -- Only returns tenant_id = 1 rows
-- Even if you do SELECT * FROM tenant_data WHERE id = <id from tenant 2>
-- It will return 0 rows due to RLS

-- Bypass RLS for admins (BYPASSRLS role attribute)
ALTER TABLE tenant_data FORCE ROW LEVEL SECURITY;
-- Even superusers are subject to RLS when FORCE is set
```

Wednesday — Backup and Recovery (90 min)

Topics:

- `pg_dump` : logical backup (SQL or custom format)
- `pg_restore` : restore from custom format dump
- `pg_dumpall` : backup entire cluster (roles + databases)
- `pg_basebackup` : physical backup for PITR
- WAL archiving and Point-in-Time Recovery (PITR)
- Backup verification: always test restores!

```
# pg_dump: logical backup
pg_dump -h localhost -U postgres -d mydb -F c -f mydb.dump
# -F c = custom format (compressed, parallel restore)

# Restore to a new database
createdb -h localhost -U postgres mydb_restore
pg_restore -h localhost -U postgres -d mydb_restore mydb.dump

# Dump specific tables
pg_dump -h localhost -U postgres -d mydb -t orders -t customers -F c -f partial.dump

# Dump as plain SQL (readable, slower restore)
pg_dump -h localhost -U postgres -d mydb -F p -f mydb.sql

# pg_dumpall: backup everything (roles, globals, all databases)
pg_dumpall -h localhost -U postgres -f cluster_backup.sql

# Physical backup with pg_basebackup
pg_basebackup -h localhost -U replication_user -D /backup/base \
    -Ft -z -P --wal-method=stream

# Parallel restore (faster for large databases)
pg_restore -h localhost -U postgres -d mydb_restore -j 4 mydb.dump
```

```
-- WAL archiving configuration (postgresql.conf)
-- wal_level = replica
-- archive_mode = on
-- archive_command = 'cp %p /archive/%f'

-- Point-in-Time Recovery (recovery.conf / postgresql.conf)
-- restore_command = 'cp /archive/%f %p'
-- recovery_target_time = '2024-11-01 14:30:00'
-- recovery_target_action = 'promote'

-- Verify backup integrity
SELECT pg_size_pretty(pg_database_size('mydb'));
-- After restore, compare sizes
```

Thursday — Configuration Tuning (60 min)

Topics:

- Memory settings: `shared_buffers`, `work_mem`, `maintenance_work_mem`
- Checkpoint settings: `checkpoint_completion_target`, `max_wal_size`
- Connection settings: `max_connections`, PgBouncer
- Parallelism: `max_parallel_workers_per_gather`
- Autovacuum tuning
- `pgtune` for baseline configuration

```
# PgBouncer configuration (pgbouncer.ini)
# [databases]
# mydb = host=localhost port=5432 dbname=mydb
#
# [pgbouncer]
# pool_mode = transaction # transaction-level pooling (best for web apps)
# max_client_conn = 1000
# default_pool_size = 25
```

```
-- Check current settings
SHOW shared_buffers;
SHOW work_mem;
SHOW max_connections;

-- Recommended baseline (adjust per server RAM):
-- shared_buffers = 25% of RAM
-- effective_cache_size = 75% of RAM
-- work_mem = RAM / max_connections / 4 (careful! per-sort, per-session)
-- maintenance_work_mem = 10% of RAM (for VACUUM, CREATE INDEX)

-- Check autovacuum settings
SELECT name, setting FROM pg_settings
WHERE name LIKE 'autovacuum%';
```

```
-- Table-specific autovacuum override (for high-churn tables)
ALTER TABLE orders SET (
    autovacuum_vacuum_scale_factor = 0.01, -- vacuum when 1% rows are dead (default 20%)
    autovacuum_analyze_scale_factor = 0.005
);
```

Friday — Security Audit + Backup Drill (45 min)

Mini-Project: Production readiness checklist.

Security Audit Checklist:

- [] No superuser connections from application
- [] All passwords use scram-sha-256 (not md5)
- [] RLS enabled on all multi-tenant tables
- [] SSL/TLS enabled for all non-local connections
- [] pg_hba.conf restricts by IP where possible
- [] Audit log enabled (pgaudit extension)

Backup Drill:

1. Take a `pg_dump` backup of your practice database
2. Drop a table
3. Restore just that table from the backup
4. Verify row count matches

PITR Drill:

1. Enable WAL archiving
2. Create a test table and insert data
3. Note the timestamp
4. Drop the table
5. Restore to the noted timestamp
6. Verify the table is back

Practice Tasks

1. Find the top 5 slowest queries using `pg_stat_statements`.
2. Identify tables with >20% dead tuple ratio and run `VACUUM VERBOSE`.
3. Set up three roles (readonly, readwrite, admin) and assign minimal permissions.
4. Enable RLS on a table and verify it works for different users.
5. Take a `pg_dump`, corrupt the database, restore from backup, and verify.
6. Install PgBouncer and configure connection pooling for your application.
7. Use `pg_basebackup` to take a physical backup.
8. Tune autovacuum for a table that receives 10,000 updates per minute.

Self-Assessment Checklist

- ☐ I can identify the top slow queries in a production database
- ☐ I understand how to set up roles with minimal privileges
- ☐ I implemented and tested Row-Level Security

- ☐ I performed a complete backup and restore cycle
 - ☐ I understand WAL archiving and PITR conceptually
 - ☐ I know the key PostgreSQL configuration parameters
 - ☐ I configured autovacuum settings for a high-churn table
-

Mock Interview Questions

1. Your PostgreSQL database is running slow. Walk me through your diagnostic process.
 2. A developer accidentally dropped a production table. How do you recover?
 3. Explain Row-Level Security and give a use case.
 4. What is the difference between `pg_dump` and `pg_basebackup` ?
 5. What does `work_mem` control? What happens if it's too low? Too high?
 6. How do you prevent connection exhaustion in a high-traffic application?
 7. Explain WAL archiving and how it enables point-in-time recovery.
 8. What is autovacuum? What happens if it can't keep up?
-

Resources

- This repo: `12_Production_PostgreSQL/` , `14_Security/` , `15_Backup_Recovery/`
- PgBouncer docs: <https://www.pgbouncer.org/config.html>
- pgtune: <https://pgtune.leopard.in.ua>
- pgaudit: <https://github.com/pgaudit/pgaudit>